

Skill_05:-

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Output for Single Quotes:-

```
v_santosh@DESKTOP-E34M7RI:~$ mkdir Skill_05.c
v_santosh@DESKTOP-E34M7RI:~$ cd Skill_05.c
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ nano Skill_05.c
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ gcc Skill_05.c -o Skill_05
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ ./Skill_05
=====
          LINUX QUOTING PARSER
=====

Supported features:
1. Single quotes
2. Double quotes
3. Spaces inside quotes
4. Variable expansion in double quotes
5. Literal variables in single quotes

Enter command:
> echo 'Hello World'

===== PARSED TOKENS =====
Token 1 : "echo" [NORMAL]
Token 2 : "Hello World" [QUOTED]
=====

===== VALIDATION =====
Result: Parsing successful.
Number of tokens: 2
Token 2 preserved as one quoted token.
=====
```

Code for Single Quotes with Variable:-

```
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ ./Skill_05
=====
          LINUX QUOTING PARSER
=====

Supported features:
1. Single quotes
2. Double quotes
3. Spaces inside quotes
4. Variable expansion in double quotes
5. Literal variables in single quotes

Enter command:
> echo '$USER'

===== PARSED TOKENS =====
Token 1 : "echo"  [NORMAL]
Token 2 : "$USER" [QUOTED]
=====

===== VALIDATION =====
Result: Parsing successful.
Number of tokens: 2
Token 2 preserved as one quoted token.
=====
```

Code for Double Quotes:-

```
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ ./Skill_05
=====
      LINUX QUOTING PARSER
=====

Supported features:
1. Single quotes
2. Double quotes
3. Spaces inside quotes
4. Variable expansion in double quotes
5. Literal variables in single quotes

Enter command:
> echo "Hello World"

===== PARSED TOKENS =====
Token 1 : "echo" [NORMAL]
Token 2 : "Hello World" [QUOTED]
=====

===== VALIDATION =====
Result: Parsing successful.
Number of tokens: 2
Token 2 preserved as one quoted token.
=====
```

Code for Double Quotes with Variable Expansion:-

```
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ ./Skill_05
=====
      LINUX QUOTING PARSER
=====

Supported features:
1. Single quotes
2. Double quotes
3. Spaces inside quotes
4. Variable expansion in double quotes
5. Literal variables in single quotes

Enter command:
> echo "Hello $USER"

===== PARSED TOKENS =====
Token 1 : "echo" [NORMAL]
Token 2 : "Hello v_santosh" [QUOTED]
=====

===== VALIDATION =====
Result: Parsing successful.
Number of tokens: 2
Token 2 preserved as one quoted token.
=====
```

Code for Multiple Quoted Tokens:-

```
v_santosh@DESKTOP-E34M7RI:~/Skill_05.c$ ./Skill_05
=====
      LINUX QUOTING PARSER
=====

Supported features:
1. Single quotes
2. Double quotes
3. Spaces inside quotes
4. Variable expansion in double quotes
5. Literal variables in single quotes

Enter command:
> echo "Hello World" 'Good Morning' Linux

===== PARSED TOKENS =====
Token 1 : "echo" [NORMAL]
Token 2 : "Hello World" [QUOTED]
Token 3 : "Good Morning" [QUOTED]
Token 4 : "Linux" [NORMAL]
=====

===== VALIDATION =====
Result: Parsing successful.
Number of tokens: 4
Token 2 preserved as one quoted token.
Token 3 preserved as one quoted token.
=====
```

Code :-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 1000
#define MAX_TOKENS 100
#define MAX_TOKEN_SIZE 200

typedef enum {
    NORMAL,
    SINGLE_QUOTE,
    DOUBLE_QUOTE
} QuoteMode;

typedef struct {
    char value[MAX_TOKEN_SIZE];
    int quoted;
} Token;

/* Add character to token */
void addChar(char token[], int *pos, char ch) {
    if (*pos < MAX_TOKEN_SIZE - 1) {
        token[*pos] = ch;
        (*pos)++;
        token[*pos] = '\0';
    }
}

/* Expand $USER and $HOME inside double quotes */
void expandVariable(char input[], char output[]) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0') {
        if (input[i] == '$') {
            if (strncmp(&input[i], "$USER", 5) == 0) {
                char *value = getenv("USER");

                if (value != NULL) {
                    strcpy(&output[j], value);
                    j += strlen(value);
                }

                i += 5;
            }
            else if (strncmp(&input[i], "$HOME", 5) == 0) {
                char *value = getenv("HOME");

                if (value != NULL) {
                    strcpy(&output[j], value);
                    j += strlen(value);
                }

                i += 5;
            }
            else {
                output[j++] = input[i++];
            }
        }
        else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}
```

```

/* Tokenize input with quote handling */
int tokenize(char input[], Token tokens[]) {

    int tokenCount = 0;
    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {

        /* Skip spaces */
        while (input[i] == ' ' ||
               input[i] == '\t') {
            i++;
        }

        if (input[i] == '\0' ||
            input[i] == '\n') {
            break;
        }

        char token[MAX_TOKEN_SIZE];
        char expanded[MAX_TOKEN_SIZE];

        int pos = 0;
        int wasQuoted = 0;

        token[0] = '\0';

        QuoteMode mode = NORMAL;

        while (input[i] != '\0' &&
               input[i] != '\n') {

            /* NORMAL MODE */
            if (mode == NORMAL) {

                /* Space ends token */
                if (input[i] == ' ' ||
                    input[i] == '\t') {
                    break;
                }

                /* Start single quote */
                if (input[i] == '\'') {
                    mode = SINGLE_QUOTE;
                    wasQuoted = 1;
                    i++;
                    continue;
                }

                /* Start double quote */
                if (input[i] == '"') {
                    mode = DOUBLE_QUOTE;
                    wasQuoted = 1;
                    i++;
                    continue;
                }

                addChar(token, &pos, input[i]);
                i++;
            }

            /* SINGLE QUOTE MODE */
            else if (mode == SINGLE_QUOTE) {

                if (input[i] == '\'') {
                    mode = NORMAL;
                    i++;
                    continue;
                }

                /*
                 * Everything inside single quotes
                 * is treated literally.
                 */
                addChar(token, &pos, input[i]);
                i++;
            }
        }
    }
}

```

```

/* DOUBLE QUOTE MODE */
else if (mode == DOUBLE_QUOTE) {

    if (input[i] == '"') {
        mode = NORMAL;
        i++;
        continue;
    }

    /*
     * Variable expansion inside
     * double quotes.
     */
    if (input[i] == '$') {

        int start = i;

        if (strncmp(&input[i],
                    "$USER", 5) == 0) {

            char *value = getenv("USER");

            if (value != NULL) {
                for (int k = 0;
                     value[k] != '\0';
                     k++) {
                    addChar(token,
                             &pos,
                             value[k]);
                }
            }

            i += 5;
        }

        else if (strncmp(&input[i],
                        "$HOME", 5) == 0) {

            char *value = getenv("HOME");

            if (value != NULL) {
                for (int k = 0;
                     value[k] != '\0';
                     k++) {
                    addChar(token,
                             &pos,
                             value[k]);
                }
            }

            i += 5;
        }

        else {
            addChar(token,
                    &pos,
                    input[i]);
            i++;
        }

        (void)start;
    }

    else {
        addChar(token,
                &pos,
                input[i]);
        i++;
    }
}
}

```

```

/* Detect unmatched quotes */
if (mode != NORMAL) {

    if (mode == SINGLE_QUOTE) {
        printf("\nError: Unclosed single quote.\n");
    }
    else {
        printf("\nError: Unclosed double quote.\n");
    }

    return -1;
}

/*
 * A quoted empty string is still a valid token.
 */
if (pos > 0 || wasQuoted) {

    strcpy(tokens[tokenCount].value, token);
    tokens[tokenCount].quoted = wasQuoted;

    tokenCount++;

    if (tokenCount >= MAX_TOKENS) {
        printf("Error: Too many tokens.\n");
        return -1;
    }
}

/*
 * Skip the space that ended this token.
 */
while (input[i] == ' ' ||
       input[i] == '\t') {
    i++;
}

return tokenCount;
}

/* Display tokens */
void displayTokens(Token tokens[], int count) {

    printf("\n===== PARSED TOKENS =====\n");

    if (count == 0) {
        printf("No tokens found.\n");
        return;
    }

    for (int i = 0; i < count; i++) {

        printf("Token %d : \"%s\"",
               i + 1,
               tokens[i].value);

        if (tokens[i].quoted) {
            printf(" [QUOTED]");
        }
        else {
            printf(" [NORMAL]");
        }

        printf("\n");
    }

    printf("===== \n");
}

```

```

/* Validate results */
void validate(Token tokens[], int count) {

    printf("\n===== VALIDATION =====\n");

    if (count == 0) {
        printf("Result: Empty command.\n");
        return;
    }

    printf("Result: Parsing successful.\n");
    printf("Number of tokens: %d\n", count);

    for (int i = 0; i < count; i++) {

        if (tokens[i].quoted) {
            printf("Token %d preserved as one quoted token.\n",
                i + 1);
        }
    }

    printf("=====\n");
}

/* Main */
int main() {

    char input[MAX_INPUT];
    Token tokens[MAX_TOKENS];

    printf("=====\n");
    printf("    LINUX QUOTING PARSER\n");
    printf("=====\n");

    printf("\nSupported features:\n");
    printf("1. Single quotes\n");
    printf("2. Double quotes\n");
    printf("3. Spaces inside quotes\n");
    printf("4. Variable expansion in double quotes\n");
    printf("5. Literal variables in single quotes\n");

    printf("\nEnter command:\n> ");

    if (fgets(input,
        sizeof(input),
        stdin) == NULL) {
        return 0;
    }

    int count = tokenize(input, tokens);

    if (count == -1) {
        return 1;
    }

    displayTokens(tokens, count);

    validate(tokens, count);

    return 0;
}

```